ID	Name
20220119	بيشوي مجدي عبدالسيد
20220143	خالد عبدالعظيم مطر
20220118	بيشوي عادل عزت
20220121	بيشوي ميلاد مكرم
20220521	ميرنا ناجح بطرس
20220350	كلارا البير حلمي

1. Project Idea in Detail:

Connect 4 is a classic strategy-based game designed for two players. The game involves a grid of 6 rows and 7 columns, suspended vertically. Players take turns dropping their uniquely colored pieces into one of the columns. The piece falls to the lowest available position in that column. The goal of the game is to align four of one's pieces in a row — horizontally, vertically, or diagonally — before the opponent does. If the grid fills up and no player has achieved this goal, the game ends in a draw. The AI agent in this project leverages the **Minimax algorithm** combined with **Alpha-Beta pruning** to make strategic decisions to be a competitive game. Using **two heuristic functions** in the Connect 4 game for enhancing the AI agent's performance and decision-making also for reducing number of states visited .

2. Main Functionalities:

1. Board Creation and Management

- Creates and visualizes the game board using **NumPy** and **Pygame**.
- The grid is drawn with cells for placing pieces.
- Updates the board dynamically as the game progresses.

```
30  # crate a 6 x 7 using zeros function to ensure that empty board is created
31  def create_board():
32      board = np.zeros((ROW_COUNT,COLUMN_COUNT))
33      return board
34
35  def print_board(board):
36      print(np.flip(board, 0))
37
38
39
40  def draw_board(board):
41      for c in range(COLUMN_COUNT):
42          for r in range(ROW_COUNT):
43              pygame.draw.rect(screen, BLUE, (c*SQUARESIZE, r*SQUARESIZE+SQUARESIZE, SQUARESIZE, SQUARESIZE))
44              pygame.draw.circle(screen, BLACK, (int(c*SQUARESIZE+SQUARESIZE/2), int(r*SQUARESIZE+SQUARESIZE+SQUARESIZE
45
46      for c in range(COLUMN_COUNT):
47          for r in range(ROW_COUNT):
48              if board[r][c] == PLAYER_PIECE:
49                  pygame.draw.circle(screen, RED, (int(c*SQUARESIZE+SQUARESIZE/2), height-int(r*SQUARESIZE+SQUARESIZE/2
50              elif board[r][c] == AI_PIECE:
51                  pygame.draw.circle(screen, YELLOW, (int(c*SQUARESIZE+SQUARESIZE/2), height-int(r*SQUARESIZE+SQUARESIZ
52      pygame.display.update()
```

(function) def print_board(board: Any) -> None

2. Win Condition Checks

- Checks if a player has aligned four pieces in any direction.
- Evaluates all possible **horizontal, vertical, and diagonal** configurations.
- Determines when the game ends with a winner

```python
def winning_move(board, piece):
    # Check horizontal locations for win / if 4 pieces for the same player are beside each other
    for c in range(COLUMN_COUNT-3):
        for r in range(ROW_COUNT):
            if board[r][c] == piece and board[r][c+1] == piece and board[r][c+2] == piece and board[r][c+3] == piece:
                return True

    # Check vertical locations for win/ if 4 pieces for the same player are on top of each other
    for c in range(COLUMN_COUNT):
        for r in range(ROW_COUNT-3):
            if board[r][c] == piece and board[r+1][c] == piece and board[r+2][c] == piece and board[r+3][c] == piece:
                return True

    # Check positively sloped diaganols / if 4 pieces for the same player are upper diagonal to each other
    for c in range(COLUMN_COUNT-3):
        for r in range(ROW_COUNT-3):
            if board[r][c] == piece and board[r+1][c+1] == piece and board[r+2][c+2] == piece and board[r+3][c+3] ==
                return True

    # Check negatively sloped diaganols / if 4 pieces for the same player are down diagonal to each other
    for c in range(COLUMN_COUNT-3):
        for r in range(3, ROW_COUNT):
            if board[r][c] == piece and board[r-1][c+1] == piece and board[r-2][c+2] == piece and board[r-3][c+3] ==
                return True
```

3. Minimax with Alpha-Beta Pruning:

- Implements AI decision-making using **Alpha-Beta Pruning** to optimize the **Minimax algorithm**.
- Balances maximizing the AI's score while minimizing the player's chances.
- Reduces unnecessary exploration of non-optimal moves.

```python
def minimax(board, depth, alpha, beta, maximizingPlayer):
    is_terminal = is_terminal_node(board)
    if depth == 0 or is_terminal:
        if is_terminal:
            if winning_move(board, AI_PIECE):
                return (None, 100000000000000)
            elif winning_move(board, PLAYER_PIECE):
                return (None, -10000000000000)
            else: # Game is over, no more valid moves
                return (None, 0)
        else: # Depth is zero
            return (None, score_position(board, AI_PIECE))
    if maximizingPlayer:
        value = -math.inf
        column = random.choice(valid_locations)
        for col in valid_locations:
            row = get_next_open_row(board, col)
            b_copy = board.copy()
            drop_piece(b_copy, row, col, AI_PIECE)
            new_score = minimax(b_copy, depth-1, alpha, beta, False)[1]
            if new_score > value:
                value = new_score
                column = col
            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return column, value

    else: # Minimizing player
        value = math.inf
        column = random.choice(valid_locations)
        for col in valid_locations:
            row = get_next_open_row(board, col)
            b_copy = board.copy()
            drop_piece(b_copy, row, col, PLAYER_PIECE)
            new_score = minimax(b_copy, depth-1, alpha, beta, True)[1]
            if new_score < value:
                value = new_score
                column = col
            beta = min(beta, value)
            if alpha >= beta:
                break
        return column, value
```

4. Game Mechanism:

- Handles dropping a piece into the correct row in the selected column.
- Ensures valid column selection and prevents illegal moves.

```python
# if it is empty in matrix or equal 0
def is_valid_location(board, col):
    return board[ROW_COUNT-1][col] == 0

def get_next_open_row(board, col):
    for r in range(ROW_COUNT):
        if board[r][col] == 0:
            return r

# place piece if it is ai or play into it
def drop_piece(board, row, col, piece):
    board[row][col] = piece

```

5. Heuristic Functions:

a) Evaluate Window

- This function evaluates a specific subset of the board (a "window") to determine how favorable it is for a given player.
- The "window" is typically a group of 4 consecutive cells (horizontal, vertical, or diagonal).
- It assigns scores based on the number of pieces for the player or the opponent in the window.

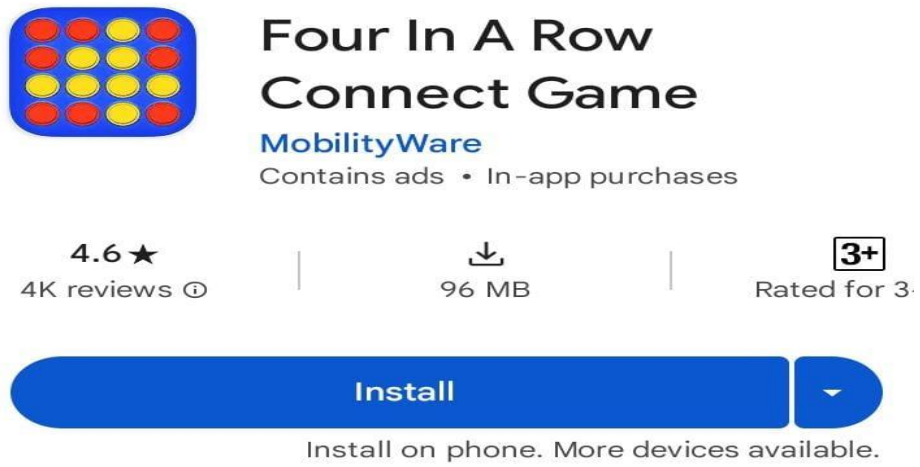
```
174  def evaluate_window(window, piece):
175      score = 0
176      opp_piece = PLAYER_PIECE
177      if piece == PLAYER_PIECE:
178          opp_piece = AI_PIECE
179
180      if window.count(piece) == 4:
181          score += 100
182      elif window.count(piece) == 3 and window.count(EMPTY) == 1:
183          score += 5
184      elif window.count(piece) == 2 and window.count(EMPTY) == 2:
185          score += 3
186      elif window.count(piece) == 1 and window.count(EMPTY) == 3:
187          score += 1
188      if window.count(opp_piece) == 3 and window.count(EMPTY) == 1:
189          score -= 10
190      return score
```

b) Score Position

- This function evaluates the entire board by applying **`evaluate window`** to all possible rows, columns, and diagonals.
- It aggregates the scores from all windows to give an overall heuristic score for the board state.
  - This function ensures the AI agent can analyze and compare different board states.

```python
def score_position(board, piece):
    score = 0

    ## Score center column
    center_array = [int(i) for i in list(board[:, COLUMN_COUNT//2])]
    center_count = center_array.count(piece)
    score += center_count * 3

    ## Score Horizontal
    for r in range(ROW_COUNT):
        row_array = [int(i) for i in list(board[r,:])]
        for c in range(COLUMN_COUNT-3):
            window = row_array[c:c+WINDOW_LENGTH]
            score += evaluate_window(window, piece)

    ## Score Vertical
    for c in range(COLUMN_COUNT):
        col_array = [int(i) for i in list(board[:,c])]
        for r in range(ROW_COUNT-3):
            window = col_array[r:r+WINDOW_LENGTH]
            score += evaluate_window(window, piece)

    ## Score posiive sloped diagonal
    for r in range(ROW_COUNT-3):
        for c in range(COLUMN_COUNT-3):
            window = [board[r+i][c+i] for i in range(WINDOW_LENGTH)]
            score += evaluate_window(window, piece)

    for r in range(ROW_COUNT-3):
        for c in range(COLUMN_COUNT-3):
            window = [board[r+3-i][c+i] for i in range(WINDOW_LENGTH)]
            score += evaluate_window(window, piece)

    return score
```

(3) Similar Applications in the Market



(4) A literature review of Academic publications (papers) relevant to the idea

- **"Connect-4 using Alpha-Beta Pruning with Minimax Algorithm"**

This paper discusses the formulation of the classic Connect-4 game utilizing Alpha-Beta Pruning with the Minimax algorithm. It examines the influence of parameters such as search depth and board size on game execution. [CloudFront]

- **"Research on Different Heuristics for Minimax Algorithm Insight from Connect-4 Game"**

This study explores how various heuristic functions can enhance the performance of the Minimax algorithm in the context of the Connect-4 game. It provides insights into the application of heuristics to improve decision-making efficiency. [Scientific Research Publishing]

- **"Alpha-Beta Pruning in Mini-Max Algorithm—An Optimized Approach for a Connect-4 Game"**

This research evaluates the extent to which Alpha-Beta pruning increases the performance of the Minimax search algorithm in Connect-4, providing an optimized approach to the game's AI implementation. [ResearchGate]

- **"Solving Connect 4 Using Optimized Minimax and Monte Carlo Tree Search"**

This paper compares the Minimax algorithm and Monte Carlo Tree Search in finding optimal moves in zero-sum games like Connect-4, offering insights into their different approaches and effectiveness. [Mililink]

- **"Evaluation of the Use of Minimax Search in Connect-4—How Does Performance Evolve on Increasing Grid Sizes"**

This study examines how the performance of the Minimax search algorithm evolves with increasing Connect-4 grid sizes, evaluating its effectiveness in making optimal moves under varying circumstances. [SCIRP]

- **"Artificial Intelligence-Based Connect 4 Player Using Python"**

This project report details the development of an AI-based Connect 4 player using Python, employing the Minimax algorithm with Alpha-Beta pruning to create a challenging opponent for human players. [Temple University Science and Technology]

6. Details of the algorithm(s)/approach(es) used :

- **Minimax Algorithm**:

The Minimax algorithm evaluates all possible moves in a game by simulating each player's turn. It chooses the move that maximizes the player's advantage while minimizing the opponent's. It recursively explores game states, returning the optimal move based on the evaluation of terminal states.

- **Alpha-Beta Pruning**:

Alpha-Beta pruning optimizes the Minimax algorithm by eliminating branches of the game tree that cannot influence the final decision. It uses two values (alpha and beta) to track the best possible scores and prunes unnecessary branches. This reduces computation time without affecting the result.

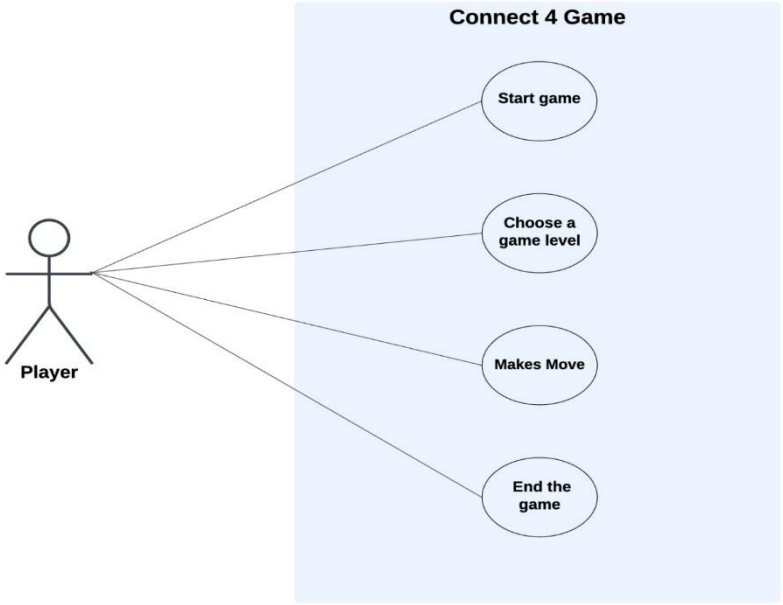
- **Heuristic Evaluation Functions**:

Heuristic functions evaluate non-terminal game states to estimate the best move when the game tree can't be fully explored. The `evaluate window` function scores individual sequences on the board, while the `score postition` function calculates a global score for the entire board. These functions guide the AI towards optimal decisions.

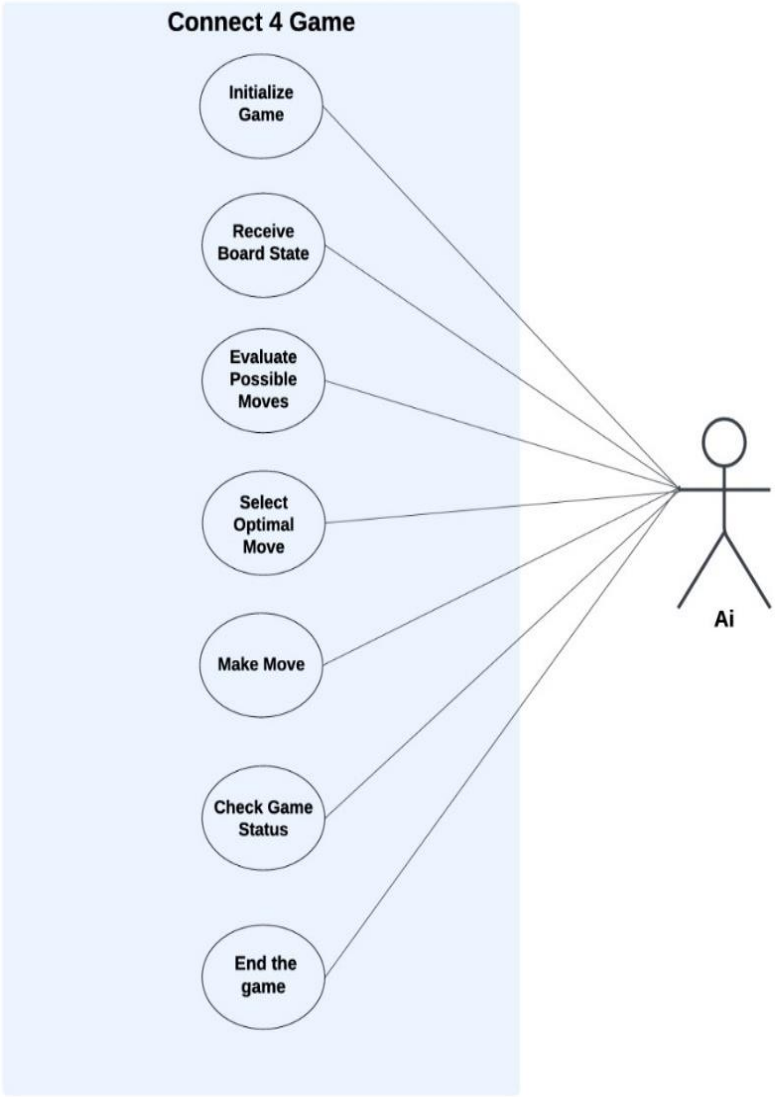
7. Diagrams:

1) Use Case

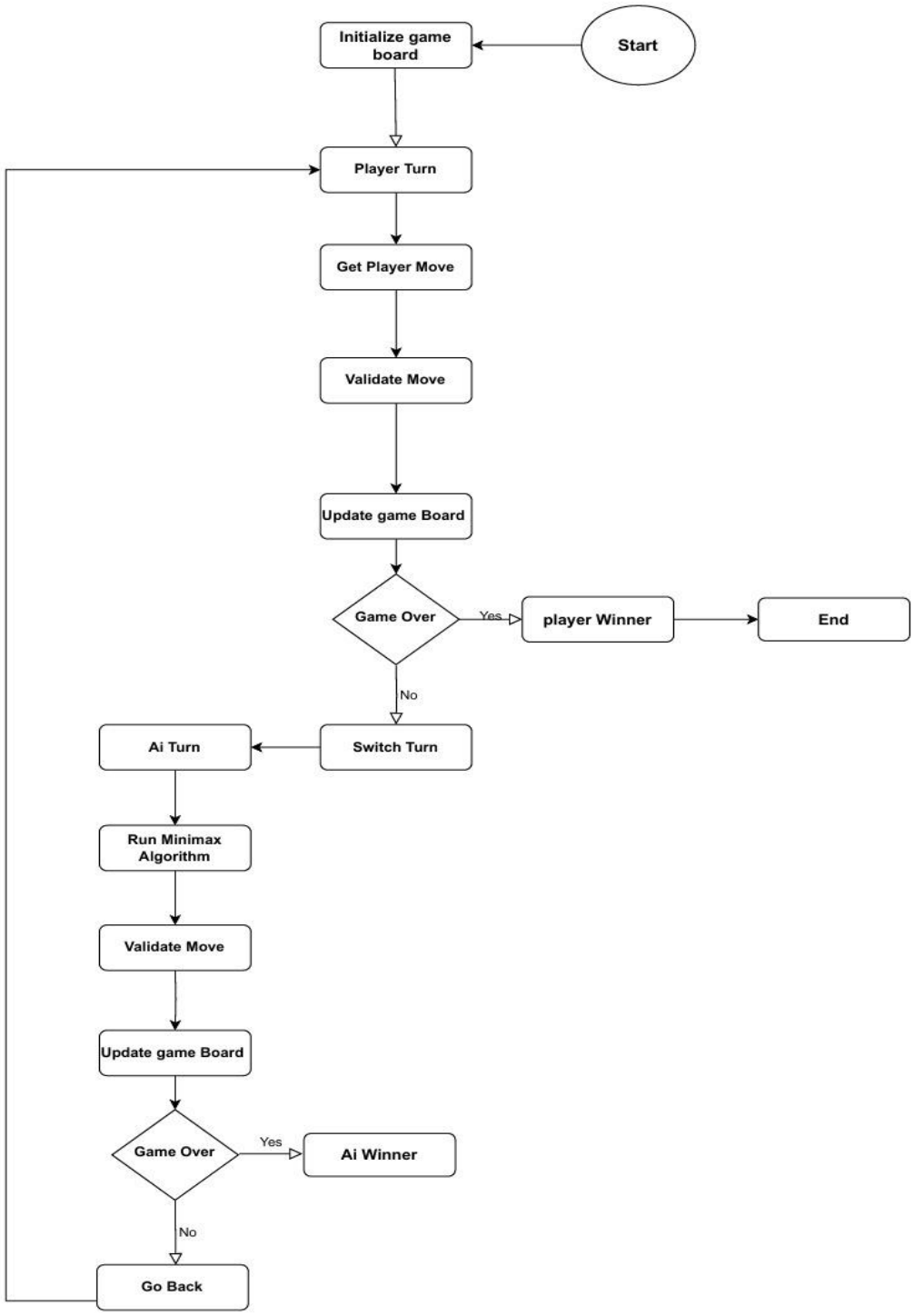
- **Player**



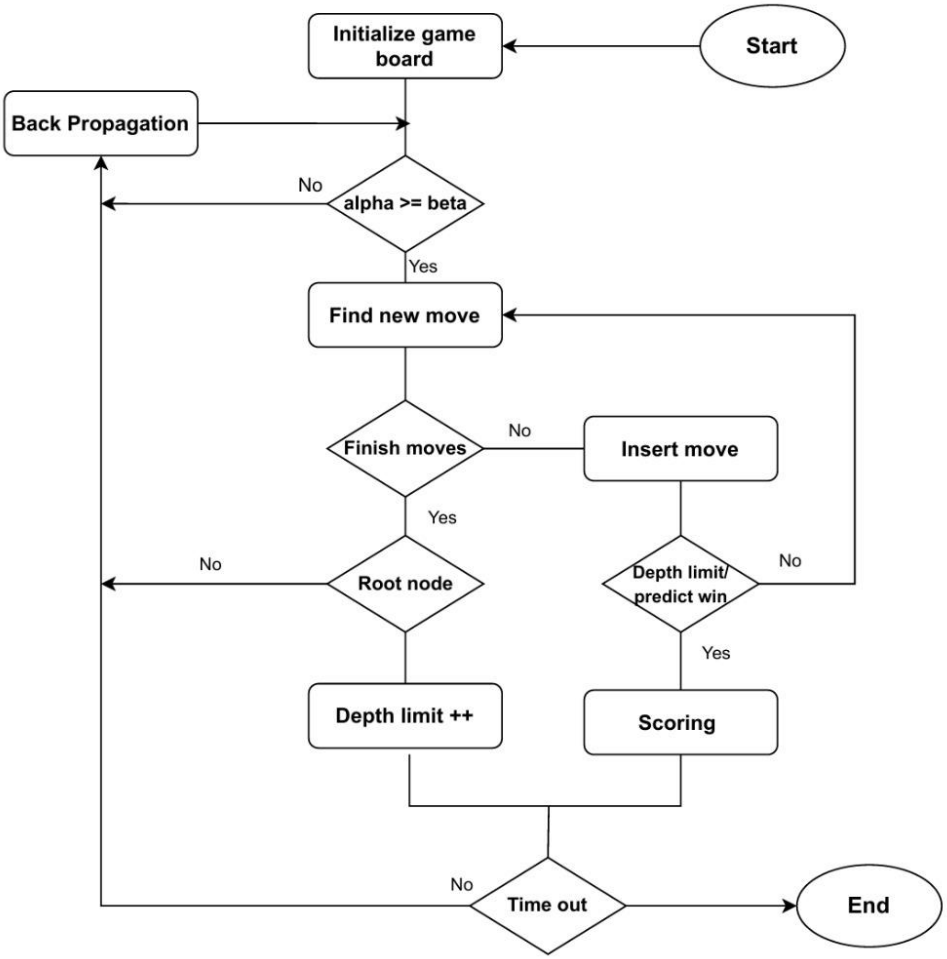
- **Ai**


Connect 4 Game
Initialize Game
Receive Board State
Evaluate Possible Moves
Select Optimal Move
Make Move
Check Game Status
End the game
Ai

2) Flowchart
General Flowchart



Alpha-Beta Flowchart



3) Block Diagram

Block Diagram: